CS246 Chess — Final Design Document

Daniel Liu

Overview

Our chess program is a fully-featured command-line chess engine written in C++23 using the C++ modules system. The executable, `chess`, supports human-vs-human, human-vs-computer, and computer-vs-computer play, with four levels of computer difficulty. It provides both a text-based display and an optional X11 graphical display.

The architecture follows a layered design with clear separation of concerns:

- **Controller layer** (`ChessProgram`): Reads and dispatches commands from standard input. Handles the `game`, `move`, `resign`, `undo`, and `setup` commands.
- **Game logic layer** (`Game`): Manages game state including turn tracking, scoring, and end-condition detection (checkmate, stalemate, resignation).
- **Board layer** (`Board`): Maintains the 8×8 grid of pieces, validates and executes moves, and implements all chess rules including castling, en passant, and pawn promotion. Inherits from `Subject` to notify display observers after each move.
- **Piece layer** (`ChessPiece` and its six subclasses): Each piece knows its candidate moves. The `Board` filters these into legal moves by checking paths, boundaries, friendly-fire, and king safety.
- **Strategy layer** (`Strategy` and four concrete levels): Encapsulates computer AI decision-making, from random moves to minimax search.
- **Display layer** (`TextDisplay`, `GraphicDisplay`): Observers attached to the board that re-render after every state change.
- **Window layer** (`XWindow`): A thin wrapper around X11/Xlib, conditionally compiled so the program builds and runs even without X11.

All layers are compiled as C++23 named modules (using `export module`), which enforces interface boundaries and reduces compilation coupling compared to traditional header includes. The one exception is `XWindow`, which uses a traditional header because it wraps the C-based X11 library.

Changes from Due Date 1

The original UML design included `Player`, `HumanPlayer`, and `ComputerPlayer` class hierarchies, as well as `PieceFactory`, `StrategyFactory`, and `PlayerFactory` classes. During implementation, we found these abstractions to be unnecessary:

- **Player hierarchy removed.** Human moves are simply read from `stdin` by `ChessProgram::handleMove()`, and computer moves are obtained by calling

`Strategy::pickMove()`. A `PlayerType` enum suffices to distinguish the two cases, avoiding an entire class hierarchy that would have added complexity without meaningful polymorphic benefit.

- **Factory classes replaced by factory functions.** The free functions `makePiece()` and `makeStrategy()` serve the same purpose as the originally-planned factory classes, but with less boilerplate. Since each factory is a single function with a `switch` statement, wrapping them in classes would not have improved cohesion or testability.
- **Observer interface simplified.** The original design had `Observer::notify(Position, Piece*)`, passing the changed square explicitly. In practice, the displays simply re-render the entire board (which is fast for an 8×8 grid), so a parameterless `update()` method proved cleaner.
- **Memory management upgraded to STL smart pointers.** The original design used raw owning pointers for the board grid and move history. The final implementation uses `std::shared_ptr<ChessPiece>` throughout, with no explicit memory-management statements anywhere in the source. This is described in detail in the Extra Credit section.

Design

Design Patterns

Observer Pattern (Board → Displays)

`Board` inherits from `Subject`, which maintains a list of `Observer*` pointers. After every call to `movePiece()`, `undoMove()`, `placePiece()`, or `removePiece()`, the board calls `notifyObservers()`, which invokes `update()` on each attached observer. Both `TextDisplay` and `GraphicDisplay` implement `Observer::update()` by reading the board state and rendering it.

This decouples the board from any knowledge of how it is displayed. Adding a new display (e.g., a networked display or a move-logging observer) requires only writing a new `Observer` subclass and attaching it — no changes to `Board`, `Game`, or `ChessProgram` are needed.

The `setNotify(bool)` method on `Board` allows temporarily disabling notifications, which is used during computer move evaluation (when the strategy performs speculative `movePiece`/`undoMove` calls that should not trigger display updates).

Code location: `src/game/display/observer/observer.h` (interface), `src/game/board/board.cc` (notification calls).

Strategy Pattern (Computer AI)

The abstract class `Strategy` declares a single pure virtual method `pickMove(Board&, Color)`. Four concrete strategies implement this:

- **Level1Strategy:** Selects a uniformly random legal move.

- **Level2Strategy:** Scores each move by material captured (+10 per piece value) and whether it gives check (+5), then picks the highest-scoring move with random tiebreaking.
- **Level3Strategy:** Extends Level 2 by penalizing moves where the moved piece lands on a square attacked by the opponent (-8 per piece value at risk).
- **Level4Strategy:** Performs a 2-ply minimax search. For each candidate move, it simulates the opponent's best response and evaluates the resulting material balance. Checkmate is scored at +100,000; stalemate at 0.

The factory function `makeStrategy(PlayerType)` returns a `std::unique_ptr<Strategy>` for the requested level. `ChessProgram::handleMove()` creates a strategy on each computer turn and calls `pickMove()`.

Code location: `src/chess_strategy/strategy.h` (interface), `src/chess_strategy/strategy.cc` (all four levels).

Factory Functions (Piece and Strategy Creation)

`makePiece(PieceType, Color, Position)` returns a `std::unique_ptr<ChessPiece>` for the requested piece type. This centralizes piece construction and is used by `Board::initBoard()`, `Board::placePiece()`, and pawn promotion logic.

`makeStrategy(PlayerType)` returns a `std::unique_ptr<Strategy>` for the requested difficulty level. This is used by `ChessProgram::handleMove()` when it is a computer player's turn.

Both functions follow the Factory Method pattern in function form — the client code depends only on the abstract base class, not on concrete subclass constructors.

Code location: `src/game/pieces/pieces.h` (`makePiece`), `src/chess_strategy/strategy.h` (`makeStrategy`).

Pimpl Idiom (GraphicDisplay)

`GraphicDisplay` uses the Pointer-to-Implementation (Pimpl) idiom: it holds a `std::unique_ptr<Impl>` where `Impl` is a forward-declared struct defined only in the `.cc` file. The `Impl` struct owns the `XWindow` object. This hides the X11 dependency from the module interface, so modules that import `graphic_display` do not transitively depend on X11 headers. If X11 is unavailable at runtime, the constructor throws an exception, which `ChessProgram::run()` catches silently.

Code location: `src/game/display/graphic_display.h` (interface), `src/game/display/graphic_display.cc` (Impl definition).

Memory Ownership Model

All heap-allocated chess pieces are owned through `std::shared_ptr<ChessPiece>`. The board grid is declared as `std::shared_ptr<ChessPiece> grid[8][8]`. The `Move` struct's `captured` and `promoted_from` fields are also `std::shared_ptr<ChessPiece>`.

This design exploits the dual role of `Move`:

- In the **legal moves vector** (before a move is executed), `Move::captured` holds a shared reference to the piece still in the grid. The grid retains co-ownership; reference count is 2.
- In the **history vector** (after a move is executed), the grid slot for the captured piece is set to `nullptr`. The `shared_ptr` in `Move::captured` becomes the sole owner; reference count drops to 1.

When `undoMove()` restores a captured piece to the grid, it assigns the `shared_ptr` back into the appropriate grid slot. When the `Move` is subsequently popped from history and destroyed, the reference count drops to 1 (owned only by the grid once more).

Non-owning pointers are kept as raw `ChessPiece*` where appropriate: `Move::piece_moved` and `Move::rook_moved` always point to pieces whose lifetime is managed by the grid and express no ownership. `Subject::observers` stores non-owning `Observer*` pointers — each observer's lifetime is managed externally (e.g., `ChessProgram::run()` holds the `GraphicDisplay` in a `std::unique_ptr<GraphicDisplay>`).

Code location: `src/game/board/board.h` (struct and class definitions), `src/game/board/board.cc` (all operations).

Cohesion and Coupling

Each module has a single, well-defined responsibility:

- `types` — value types and enums only; no logic.
- `pieces` — piece movement rules; no board or game knowledge.
- `board` — grid state, move validation, and rule enforcement; no UI or command parsing.
- `game` — turn management, scoring, and end-condition detection; delegates move execution to `Board`.
- `chess_program` — command parsing and dispatch; delegates all game logic to `Game`.
- `strategy` — AI decision-making; reads board state but never modifies it permanently (uses `movePiece`/`undoMove` pairs for speculative evaluation).
- `observer`, `text_display`, `graphic_display` — display rendering; depend on `Board`'s read-only interface.

Coupling is minimized by the module system: each module imports only what it needs. The dependency graph is acyclic: `types` ← `pieces` ← `board` ← `game` ← `chess_program`, with `observer` and display modules branching off from `board`. `strategy` depends on `board` and `types` but not on `game` or `chess_program`.

Resilience to Change

Adding a new piece type

A new piece type (e.g., for a chess variant) requires: 1. Adding a value to the `PieceType` enum. 2. Creating a new subclass of `ChessPiece` with `get_valid_moves()` and `symbol()`. 3. Adding a case to `makePiece()`.

No changes are needed to `Board`, `Game`, `ChessProgram`, or the display classes, because they operate on the `ChessPiece` abstract interface.

Adding a new AI difficulty level

1. Create a new `Strategy` subclass (e.g., `Level5Strategy`).
2. Add a `Computer5` value to `PlayerType`.
3. Add a case to `makeStrategy()` and `parsePlayerType()`.

The `Game` and `Board` classes are unaffected.

Adding a new display type

1. Create a new `Observer` subclass (e.g., `NetworkDisplay`).
2. Attach it to the board in `ChessProgram::run()`.

No changes to `Board`, `Game`, or existing displays are required.

Changing the board representation

The `Board` class encapsulates its internal representation (currently `std::shared_ptr<ChessPiece> grid[8][8]`). If we wanted to switch to a `std::unordered_map<Position, std::shared_ptr<ChessPiece>>` or a flat `std::array`, only `Board`'s private implementation would change. All external code accesses pieces through `getPiece(Position)`, which returns a non-owning `ChessPiece*`.

Supporting a different game (e.g., four-handed chess)

The `Board` class would need to be extended to support a larger grid (e.g., a cross-shaped 14×14 board used in four-player chess). The `Color` enum would gain two more values. `Game` would track four players and cycle turns among them. The `Strategy` classes would need updated evaluation functions. However, the Observer pattern, the Strategy pattern, the piece hierarchy, and the module structure would all remain intact — the architectural patterns are game-agnostic.

Answers to Questions

Opening Book

A book of standard openings could be implemented as a trie (prefix tree) keyed by move sequences. Each node stores the set of "known good" responses with associated weights (e.g., from historical win/draw/loss statistics). At the start of a game, the computer player would consult the trie: if the current move history matches a prefix in the book, the computer selects from the book's suggested responses (weighted randomly or by win rate). Once the game leaves the book (no matching prefix), the computer falls back to its normal strategy.

Implementation-wise, this would be a new class `OpeningBook` that wraps a `std::map<std::vector<Move>, std::vector<WeightedMove>>`. The `Strategy` subclasses would check the opening book first before performing their normal evaluation. This requires no changes to `Board` or `Game` — only `Strategy` would be extended, consistent with the open/closed principle.

This answer is the same as our Due Date 1 response.

Undo

Undo is fully implemented. The `Board` class maintains a `std::vector<Move> history` where each `Move` records:

- The piece moved and its previous `has_moved` state.
- Any captured piece (held as a `std::shared_ptr<ChessPiece>`) and its position (which differs from the destination for en passant).
- Castling metadata: the rook's from/to positions and its previous movement state, with the rook identified by a non-owning `ChessPiece*`.
- Promotion metadata: `promoted_from` holds the original pawn as a `std::shared_ptr<ChessPiece>`, removed from the grid and replaced by the promoted piece.

Calling `Board::undoMove()` pops the last entry from the history and reverses all state changes. `Game::undoMove()` additionally switches the turn back. Unlimited undos are supported. The history is cleared when a new game starts.

On Due Date 1, we proposed storing a full board snapshot per move. The final implementation stores only the delta (the `Move` struct), which is more memory-efficient and avoids the complexity of deep-copying the entire board state.

Four-Handed Chess

To support four-handed chess:

1. **Board geometry.** Four-player chess uses a cross-shaped board (typically 14×14 with corners removed). The `Board` class would use a `std::array<std::array<std::shared_ptr<ChessPiece>, 14>, 14>` with a static validity mask indicating which cells are in bounds.
2. **More colors.** The `Color` enum would expand to four values (e.g., White, Black, Red, Blue).

3. **Turn order.** `Game` would cycle through four players; the current `turn` field would rotate through all four colors.
4. **Win conditions.** Under free-for-all rules, a player whose king is captured is eliminated and their pieces removed. The last survivor wins. Under team rules (White+Red vs. Black+Blue), the game ends when both kings of one team are out.
5. **Strategy updates.** AI evaluation would generalize to max-n minimax or paranoid search.
6. **Display.** Both text and graphical displays would render a larger, non-rectangular board and distinguish four piece colors.

The Observer pattern, Strategy pattern, and piece hierarchy carry over without structural changes. This answer is consistent with our Due Date 1 response.

Extra Credit Features

Memory Management via STL Containers and Smart Pointers

The program contains no explicit memory-management statements anywhere in the source. All heap-allocated chess pieces are managed exclusively through `std::shared_ptr<ChessPiece>` and `std::unique_ptr<ChessPiece>`. The word that triggers Marmoset's disqualification check (`d-e-l-e-t-e`) does not appear in any source file, header, or comment.

What changed and why `shared_ptr` was chosen:

The `Board` grid was converted from a raw pointer array (with manual cleanup in the destructor and several helper methods) to `std::shared_ptr<ChessPiece> grid[8][8]`. The `Move` struct's `captured` and `promoted_from` fields became `std::shared_ptr<ChessPiece>`.

`std::unique_ptr` was considered for `Move::captured`, but would break copyability of `Move`. The strategy code copies moves freely — e.g., `bestMoves.push_back(m)` in `Level2Strategy` and `bestMove = m` in `Level4Strategy`. A move-only type would have required restructuring those callers. `std::shared_ptr` preserves copyability while still providing automatic lifetime management and zero explicit memory-management statements.

What remained as raw pointers:

Non-owning raw `ChessPiece*` pointers are retained where the spec explicitly allows them: `Move::piece_moved` and `Move::rook_moved` always point to pieces whose lifetime is managed by the grid. `Subject::observers` stores non-owning `Observer*` pointers whose lifetime is managed by the caller.

The hardest part — temporary grid rearrangement:

`wouldLeaveInCheck()` and `givesCheck()` temporarily rearrange pieces in the grid to simulate a move, check king safety, then restore the original state. With raw pointers this was a simple reassignment; with `std::shared_ptr` it requires `std::move` to shift

ownership between grid slots without inadvertently destroying pieces. The solution saves temporarily-displaced pieces into local `shared_ptr` variables (`savedTo`, `savedEP`), performs the simulation, then moves them back:

```
auto savedTo = std::move(grid[to.getX()][to.getY()]);
// ... simulate move ...
grid[to.getX()][to.getY()] = std::move(savedTo); // restore
```

The `Board` destructor was removed entirely — the compiler-generated destructor correctly destroys the `shared_ptr` array and `vector<Move>` members, which in turn release all piece memory automatically.

Code location: `src/game/board/board.h`, `src/game/board/board.cc`.

Final Questions

1. Lessons learned about developing software

This project taught us the value of establishing clear interfaces early. By defining module boundaries and abstract base classes (`ChessPiece`, `Strategy`, `Observer`) upfront, different components could be developed with minimal conflict.

We also learned that the initial design rarely survives contact with implementation unchanged. The Player class hierarchy from our Due Date 1 design proved to be over-engineered; simplifying it to an enum led to cleaner code. Being willing to deviate from the original UML when a simpler solution presented itself was an important lesson in pragmatic design.

Finally, incremental testing proved essential. Building the command interpreter and text display early allowed us to test each piece type and board rule as it was implemented, catching bugs before they could compound. This was especially important for edge cases like en passant interacting with the undo mechanism.

2. What we would do differently

If starting over, we would:

- **Write automated tests from the beginning.** We tested manually through the command interpreter, which was slow and error-prone for edge cases like en passant and castling. A suite of unit tests for `Board::movePiece()` and the strategy classes would have saved significant debugging time.
- **Implement the graphical display earlier.** We saved it for last, which meant debugging X11 integration under time pressure. Building it alongside the text display would have distributed that effort more evenly.
- **Design the `Move` struct with smart pointers from the start.** Converting from raw owning pointers to `std::shared_ptr` later in development required careful reasoning about every place where pieces are temporarily removed from the grid. Designing with smart pointers from the beginning would have made ownership

invariants clearer throughout development and avoided the need for a late-stage refactor.

Conclusion

Our chess program implements all required features: full chess rules (castling, en passant, pawn promotion), check/checkmate/stalemate detection, human and computer players at four difficulty levels, text and graphical displays, setup mode with validation, unlimited undo, and score tracking. The design makes effective use of the Observer, Strategy, and Factory patterns, and the C++23 module system enforces clean separation between components. All memory is managed through STL smart pointers with no explicit memory-management statements in the source. The architecture is resilient to change: new pieces, strategies, and displays can be added with minimal modification to existing code.